Boogle Dox — Architecture & Engineering Decisions

Status: Proposed architecture; update after implementation to reflect the actual code.

Technical approach

I chose a JavaScript-based web stack because Boogle Dox is designed to run directly in a browser. JavaScript is suited to interactive editor controls, formatting, menus, and immediate feedback. I have worked with React and Node.js before, so using a familiar ecosystem lets me spend the timebox on product behavior rather than learning a new framework. My preferred implementation is **Next.js with TypeScript**: TypeScript is JavaScript with static type checking, which can help catch mismatched document and permission data while I develop. I would not describe this as plain JavaScript if the final code uses TypeScript.

• **Frontend:** Next.js/React and TypeScript; Tailwind CSS for consistent styling.

• **Editor:** Tiptap (ProseMirror-based) to handle rich text and serialize content as JSON rather than building editing behavior from scratch.

• **Backend:** Next.js server routes/actions for document and sharing operations.

• **Persistence:** hosted Supabase PostgreSQL so content and permissions survive refresh and can be tested on a public deployment.

• **Authentication:** Supabase Auth, or explicitly documented seeded demo accounts if setup requires simpler access.

• **Deployment:** Vercel for the web app, backed by a hosted database. Free tiers should be sufficient for evaluation, subject to current service limits.

• **Testing:** a focused automated test for permissions, plus manual end-to-end verification of the main flow.

Suggested data model

`users`: id, email, display_name

`documents`: id, owner_id, title, content_json, revision, created_at, updated_at

`document_access`: document_id, user_id, role ('editor' or 'viewer'), granted_by, created_at

`edit_sessions` (stretch only): document_id, user_id, lease_expires_at

The owner is stored on the document itself. Access rows represent other people who have been granted permissions. A revision number helps detect stale updates.

Backend authorization

Every document read checks whether the current user is the owner or has a share grant. Every write checks for owner/editor permissions, and changing permissions requires the owner. This must happen server-side or through correctly configured database policies, not only through disabled buttons in React. A copied URL must never bypass the authorization check.

Persistence and save behavior

Tiptap content is stored as structured JSON so formatting can be reopened. Save operations should validate the title/content payload and use the document revision to avoid silently overwriting newer changes. The UI should clearly show saving, saved, or failed states. If I implement debounced autosave, I will also protect against overlapping saves; if not, I will ship an explicit Save button and explain that choice.

Import and validation

The initial importer supports TXT and MD files, with a stated size limit (to be defined in code). I will validate the file size, extension and contents and show an actionable error for unsupported files. Markdown parsing should use a trusted parser with safe handling; plain text must not be inserted as untrusted HTML.

Controlled editing — future enhancement

My preferred collaboration model is a server-enforced editing lease. One user actively edits while other authorized users can view the last saved revision and see an editing indicator. When the editor selects Save & Finish, the server persists changes, releases the lease, and other clients can refresh or receive a status notification. A timeout and conflict handling are necessary so closing a browser tab does not lock a document forever. This is a stretch feature, not a substitute for basic sharing.

Known tradeoffs

• Rich-text library instead of hand-written contentEditable logic: reliability and speed.

• Structured editor JSON instead of raw HTML: preservation and safer rendering.

• Server-checked roles instead of UI-only permissions: security and testability.

• Simple page visual and ruler rather than a full pagination engine: visual familiarity without scope explosion.

• No simultaneous collaboration CRDT/OT engine in the initial timebox: avoids high implementation and conflict risk.

Tests and evidence to record after building

• [ ] Create, rename, format, save, refresh, reopen.

• [ ] Import supported file; reject unsupported/oversize file.

- [ ] Viewer can open but cannot modify content by direct server request.
- [ ] Unauthorized account cannot open a private URL.
- [ ] Owner can add/remove sharing grants.
- [ ] Stale document revision cannot overwrite a new revision.
- [ ] Hosted URL works in a clean browser session.

Important: Checkboxes are intentionally not checked; they are not claims of completed testing.